SIMATS ENGINEERING

ASSESSMENT TOOL 2: Case Study

COURSE NAME: Cloud Computing and Big Data Analytics **COURSE CODE:** CSA15

COURSE OUTCOME COVERED

CO3: *Implement cloud-based solutions using cloud storage, web APIs, and platform services for real-world applications.* (BL3)

Dave's Taxonomy: Precision (Level 3) **Total Marks: 30**

Question Count: 3

Code of Conduct

I NITHISH KUMAR K(192521266) (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.

I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student: knithishkumar

Assessment Tasks

Case Study

1. **Case Study 1: Smart Retail Inventory Management System**

   A retail chain implements a cloud-based inventory system connected to store devices.Clients (POS systems and dashboards) communicate via APIs to centralized cloud services. Cloud storage stores product data, transaction logs, and analytics datasets. The system uses platform services for analytics and demand forecasting. Secure access is maintained through VPNs, firewalls, and API gateways. This solution integrates infrastructure, services, and web applications for efficient operations.

2. **Case Study 2: Cloud-Based Document Collaboration System**

   An enterprise deploys a document collaboration tool similar to Google Docs. Users access the

platform via browsers, enabling real-time editing and sharing. Documents are stored in distributed cloud storage with version control mechanisms. Web APIs enable synchronization across multiple clients and devices.

Network infrastructure ensures low latency and high availability globally. Security includes encryption, identity management, and access permissions for collaboration.

3. **Case Study 3: Online Food Ordering Platform**

A startup builds a cloud-based food ordering system accessible via web browsers and mobile clients.

Frontend clients interact with backend services through RESTful Web APIs hosted on a cloud platform.

Cloud storage is used to store menu images, invoices, and customer data securely. The infrastructure uses virtual machines and managed Kubernetes for scalability and reliability. Security is enforced through authentication tokens, HTTPS, and role-based access control. The system demonstrates integration of clients, APIs, storage, and platform services in real time.

Case Study Rubric

Criteria	**Weightage**	**Level 4 - Exemplary**	**Level 3 - Proficient**	**Level 2 - Developing**	**Level 1 - Beginning**
Understanding of Case	25%	Clearly identifies all technical and business requirements	Identifies most requirements	Basic understanding only	Poor understanding of case
Application of Concepts	25%	Applies suitable cloud concepts and tools effectively	Good application with minor issues	Partial application	Weak or incorrect application
Analysis	20%	Strong analysis with clear trade-off reasoning	Reasonable analysis	Limited analysis	Minimal analysis
Solution	20%	Solution is	Reasonable	Basic solution	Unclear

Design		feasible, practical, and well justified	solution with minor gaps		solution
Clarity	10%	Well-structured and easy to follow	Mostly clear	Somewhat unclear	Poorly organized

CASE STUDY 1: SMART RETAIL INVENTORY MANAGEMENT SYSTEM

1. Introduction

A retail chain requires a cloud-based inventory management system that connects multiple stores, POS systems, dashboards, centralized cloud services, storage, analytics, and security infrastructure. The main objective is to provide real-time inventory visibility, improve stock management, support demand forecasting, and ensure secure and scalable operations.

2. Business Requirements

The major business requirements are:

- Real-time inventory tracking across multiple stores.
- Integration with POS systems.
- Centralized management of product and transaction data.
- Real-time synchronization between stores and the central system.
- Demand forecasting using historical transaction data.
- Analytics and reporting for management.
- High availability and scalability during peak sales periods.
- Secure access to inventory and transaction information.
- Reduction of physical infrastructure and maintenance costs.
- Easy expansion when new stores are added.

3. Technical Requirements

The system requires:

- POS systems and store devices as clients.
- Web-based management dashboards.

RESTful APIs for communication.

API Gateway for controlling API requests.

Cloud-based databases for structured product and transaction data.

Cloud storage for large datasets and files.

Cloud analytics services.

Machine learning services for demand forecasting.

VPN and firewall for secure network connectivity.

Identity and access management.

Load balancing and auto-scaling.

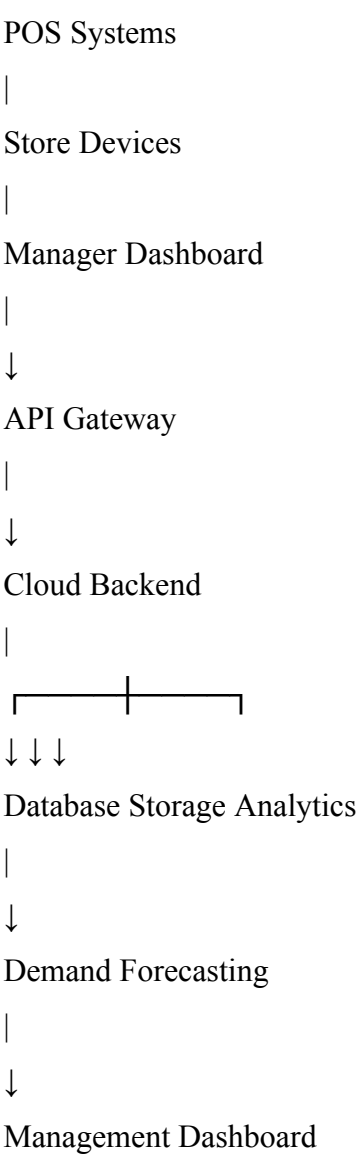
Monitoring and logging services.

4. Cloud Computing Concepts Used

Requirement	Cloud Technology/Concept
POS communication	RESTful Web APIs
API management	API Gateway
Product and transaction data	Cloud Database
Large datasets	Cloud Storage
Demand forecasting	Cloud ML/AI Services
Business analytics	PaaS Analytics Services
Secure store connectivity	VPN
Network protection	Firewall

Secure API access	API Gateway and Authentication
Scalability	Auto-scaling
Application hosting	Cloud Infrastructure
Management interface	Web Application

5. Proposed Architecture



6. Explanation of Architecture

The POS systems and store devices send inventory and transaction information to the cloud through APIs. The API Gateway acts as the secure entry point for these requests.

The cloud backend processes the requests and stores structured information such as product details, stock levels, and transactions in a cloud database.

Cloud storage can be used for large datasets and files. Analytics services process historical transaction information and generate useful business insights.

Machine learning services can analyze historical sales patterns to forecast future demand. This allows the retail chain to maintain appropriate stock levels and reduce both overstocking and stock shortages.

7. Security

Security is an important requirement because the system handles business and transaction data.

The following security mechanisms can be implemented:

VPN for secure communication between stores and the cloud.

Firewalls for protecting cloud infrastructure.

API Gateway for controlling API access.

Authentication for verifying users and devices.

Role-Based Access Control (RBAC) for restricting access based on user roles.

Encryption for protecting sensitive information.

Logging and monitoring for detecting suspicious activities.

8. Scalability and Availability

The system should support additional stores and increasing transaction volumes without major architectural changes.

Cloud auto-scaling can automatically increase or decrease computing resources depending on demand.

For example:

Normal traffic:
100 requests/minute → 3 application instances

Peak traffic:
10,000 requests/minute → Auto-scaling → Multiple application instances

Load balancers can distribute requests across multiple application instances, improving availability and performance.

9. Advantages

Centralized inventory management.

Real-time visibility of stock.

Improved demand forecasting.

Easy scalability.

Reduced physical infrastructure requirements.

Better analytics and decision-making.

Improved security.

Easy integration of new stores.

10. Trade-offs

Although cloud computing provides many advantages, there are some trade-offs

Dependence on Internet connectivity can affect store operations.

Continuous cloud usage can increase operational costs.

Sensitive business data requires strong security controls.

API or cloud service failures may temporarily affect POS operations.

Cloud-based analytics and machine learning require additional technical expertise.

11. Conclusion

A cloud-based smart retail inventory management system provides centralized, scalable, secure, and intelligent inventory management. The combination of APIs, cloud storage, databases, analytics, machine learning, VPNs, firewalls, and API gateways makes the solution suitable for a large retail chain. The architecture can scale as the number of stores and transactions increases.

CASE STUDY 2: CLOUD-BASED DOCUMENT COLLABORATION SYSTEM

1. Introduction

A cloud-based document collaboration system allows users to create, edit, share, and manage documents through browsers and other devices. Multiple users can work on the same document in real time. Cloud storage, APIs, distributed infrastructure, encryption, identity management, and access permissions are used to provide a secure and highly available collaboration platform.

2. Business Requirements

The major business requirements are:

Users should access documents from anywhere.

Multiple users should edit documents simultaneously.

Changes should be synchronized in real time.

Documents should be securely stored.

Previous versions should be available.

Users should be able to share documents.

Different access permissions should be supported.

The system should provide high availability.

The system should provide low latency for global users.

The system should support a large number of users.

3. Technical Requirements

The system requires:

Web browsers and mobile clients.

Web application.

Real-time communication APIs.

Web APIs for synchronization.

Distributed cloud storage.

Version control mechanisms.

Authentication services.

Identity management.

Access control.

Encryption.

Load balancing.

Global cloud infrastructure.

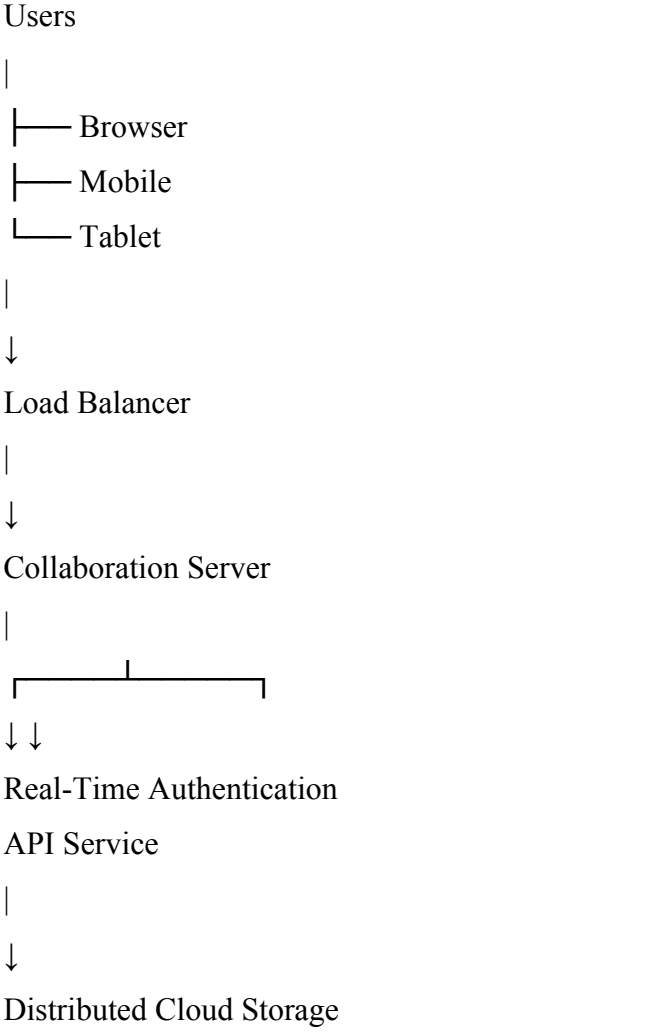
Monitoring and logging.

4. Cloud Computing Concepts Used

Requirement	Cloud Technology/Concept
User access	Web Application
Document storage	Distributed Cloud Storage
Real-time editing	WebSockets / Real-Time APIs

Synchronization	Web APIs
Version history	Version Control
Authentication	Identity and Access Management
Data protection	Encryption
Permission management	Access Control / RBAC
Global availability	Distributed Cloud Infrastructure
Scalability	Auto-scaling
Traffic distribution	Load Balancer

5. Proposed Architecture



|
├── Current Document
├── Previous Version
├── Version History
└── Shared Documents

6. Explanation of Architecture

Users access the system through browsers or mobile devices. Requests are sent through a load balancer, which distributes traffic among multiple application servers.

The collaboration server manages document editing and communication between users.

Real-time APIs allow changes made by one user to be transmitted to other users immediately.

Documents are stored in distributed cloud storage. Version control mechanisms maintain previous versions of documents, allowing users to recover older versions when required.

Identity management services authenticate users, while access-control mechanisms determine which documents each user can view or edit.

7. Real-Time Collaboration

One of the biggest technical challenges is simultaneous editing.

For example:

User A → Edits Paragraph 1
User B → Edits Paragraph 1
↓
Potential Conflict
↓
Conflict Resolution

Technologies such as Operational Transformation (OT) or Conflict-Free Replicated Data Types (CRDTs) can be used to manage concurrent changes and maintain document consistency.

8. Security

The following security mechanisms should be implemented:

Encryption during data transmission

Encryption for stored documents.

Strong user authentication.

Identity management.

Role-Based Access Control.

Document-level permissions

Secure APIs.

Monitoring and logging.

For example:

Owner → Read + Write + Share

Editor → Read + Write

Viewer → Read Only

9. Scalability and Availability

The system should be able to support users from different geographical locations.

Distributed cloud infrastructure can reduce latency by placing services closer to users.

Load balancing distributes requests among multiple servers.

Auto-scaling can automatically increase resources when the number of users increases.

10. Advantages

Real-time collaboration.

Global accessibility.

Automatic document backup.

Version history

Easy sharing.

High availability.

Scalability.

Reduced dependency on local storage.

11. Trade-offs

Real-time synchronization is technically complex.

Concurrent editing can create consistency problems.

Maintaining multiple document versions requires additional storage.

Sensitive documents require strong security.

Distributed systems require careful synchronization.

Global infrastructure can increase operational costs.

12. Conclusion

A cloud-based document collaboration system provides real-time document editing, centralized storage, version management, secure sharing, and global accessibility. Distributed cloud infrastructure combined with real-time APIs, identity management, encryption, and access control provides a scalable and reliable collaboration platform.

CASE STUDY 3: ONLINE FOOD ORDERING PLATFORM

1. Introduction

An online food ordering platform allows customers to browse menus, place orders, and manage their accounts using web browsers and mobile applications. The frontend communicates with cloud-hosted backend services through RESTful Web APIs. Cloud storage, virtual machines, Kubernetes, authentication tokens, HTTPS, and Role-Based Access Control are used to provide scalability, security, and reliability.

2. Business Requirements

The major business requirements are:

Customers should browse restaurant menus.

Customers should place food orders online.

Restaurants should manage menus and orders.

Customers should receive order information.

The system should support web and mobile clients.

Customer information should be stored securely.

Menu images and invoices should be stored.

The system should support large numbers of simultaneous users.

The platform should remain available during peak hours.

The system should be scalable and reliable.

3. Technical Requirements

The system requires:

Web frontend

Mobile application.

RESTful Web APIs.

API Gateway.

Cloud backend services.

Virtual machines.

Containerized applications.

Managed Kubernetes cluster.

Cloud database.

Object/cloud storage.

Authentication tokens.

HTTPS.

Role-Based Access Control.

Load balancing.

Auto-scaling.

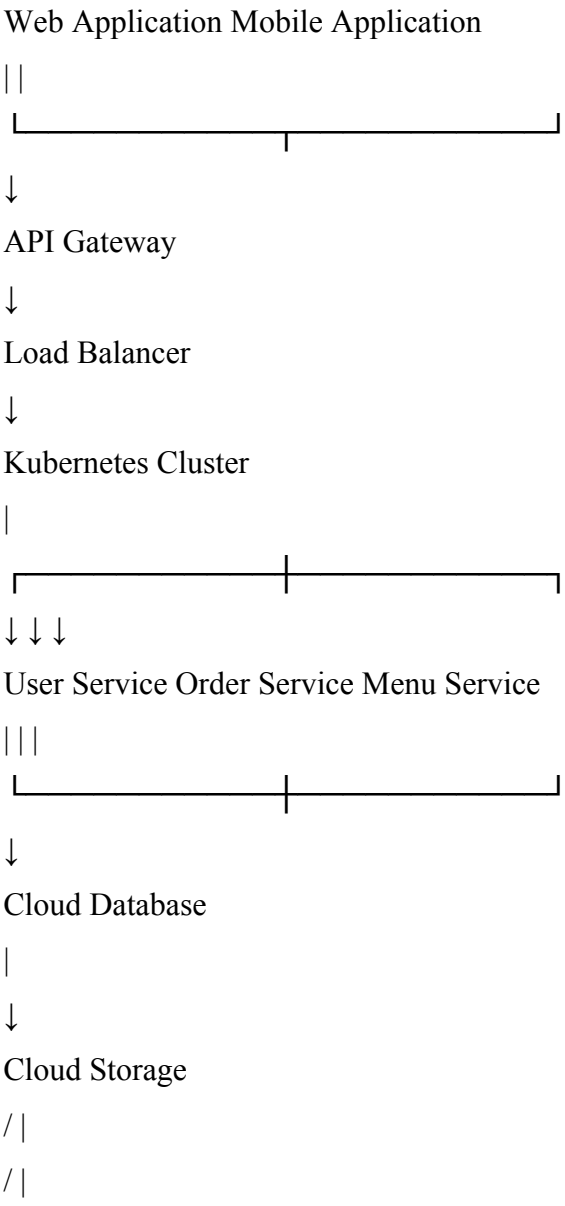
Monitoring and logging.

4. Cloud Computing Concepts Used

Requirement	Cloud Technology/Concept
Web and mobile clients	Client Applications
Backend communication	RESTful Web APIs
Application hosting	Cloud Virtual Machines
Container management	Kubernetes
Menu images	Cloud Object Storage
Customer data	Cloud Database

Authentication	Authentication Tokens
Secure communication	HTTPS
Authorization	RBAC
Scalability	Kubernetes Auto-scaling
Reliability	Multiple Application Instances
Traffic distribution	Load Balancer

5. Proposed Architecture



Images Invoices Files

6. Explanation of Architecture

Customers access the food ordering system using a web browser or mobile application.

The frontend sends requests to the backend through RESTful Web APIs. The API Gateway provides a controlled entry point for these requests.

A load balancer distributes incoming traffic across multiple application instances.

The backend services are deployed inside a managed Kubernetes cluster. Separate services can be created for users, orders, menus, and other functions.

Structured information such as customer details, restaurant information, menu details, and order information can be stored in a cloud database.

Menu images, invoices, and other files can be stored in cloud object storage.

7. Role of Kubernetes

Kubernetes is used to manage containerized backend services.

During normal traffic:

100 users/minute
↓
3 application containers

During peak traffic:

10,000 users/minute
↓
Kubernetes Auto-scaling
↓
Multiple application containers

Kubernetes can automatically manage container deployment, scaling, service discovery, and recovery.

This makes the system suitable for peak periods such as lunch, dinner, weekends, and special offers.

8. Security

Security can be implemented using:

Authentication tokens to identify users.

HTTPS to encrypt communication.

Role-Based Access Control to restrict operations.

Secure API endpoints.

Encrypted cloud storage.

Secure database access.

Monitoring and logging.

Example:

Customer → Customer APIs

Restaurant → Restaurant APIs

Administrator → Administrative APIs

A customer should not be allowed to access restaurant-management or administrator functions.

9. Scalability and Reliability

The platform should handle sudden increases in traffic.

For example, during normal hours:

Low traffic → Fewer containers

During peak hours:

High traffic → More containers

After peak hours:

Low traffic → Resources reduced

This approach prevents the system from maintaining unnecessary resources when demand is low.

Multiple application instances can also improve reliability because if one instance fails, requests can be redirected to another instance.

10. Advantages

High scalability.

Support for web and mobile applications.

Independent backend services.

Efficient storage of images and invoices

High availability

Automatic scaling.

Secure communication.

Easy deployment of new services.

Kubernetes provides container management and recovery.

11. Trade-offs

Kubernetes introduces additional complexity.

Cloud infrastructure costs increase with high traffic.

Microservices require proper monitoring.

Multiple services create additional points of failure.

Kubernetes requires specialized technical knowledge.

Improper configuration can increase infrastructure costs.

12. Conclusion

The proposed cloud-based food ordering platform integrates web and mobile clients, RESTful APIs, cloud storage, virtual machines, Kubernetes, authentication tokens, HTTPS, and RBAC. The architecture provides scalability, reliability, security, and flexibility. Kubernetes allows the platform to automatically scale according to demand, making it suitable for real-time food ordering applications.

COMPARISON OF ALL THREE CASE STUDIES

Factor	Smart Retail	Document Collaboration	Food Ordering
Main Objective	Inventory Management	Real-Time Collaboration	Online Food Ordering
Main Clients	POS and Dashboards	Browsers and Devices	Web and Mobile
Main API	Inventory APIs	Synchronization APIs	REST APIs
Storage	Product and Transaction Data	Distributed Documents	Images, Invoices, Customer Data
Main Challenge	Demand Forecasting	Concurrent Editing	Scalability
Key Technology	Analytics and ML	OT/CRDT and Real-Time APIs	Kubernetes
Security	VPN, Firewall, API Gateway	Encryption, IAM	HTTPS, Tokens, RBAC
Scalability	Auto-scaling	Distributed Infrastructure	Kubernetes
Cloud Approach	IaaS + PaaS + SaaS	PaaS + SaaS	IaaS + PaaS

OVERALL ANALYSIS

All three case studies demonstrate how cloud computing can integrate infrastructure, platforms, applications, APIs, storage, networking, and security.

The Smart Retail Inventory Management System focuses mainly on centralized inventory management, analytics, and demand forecasting.

The Cloud-Based Document Collaboration System focuses on distributed storage, real-time synchronization, concurrent editing, and global availability.

The Online Food Ordering Platform focuses on RESTful APIs, cloud application hosting, Kubernetes, scalability, authentication, and role-based access control.

Among the three systems, the Online Food Ordering Platform provides a strong example of cloud computing because it combines multiple cloud concepts including Infrastructure as a Service, Platform as a Service, containers, Kubernetes, REST APIs, cloud storage, authentication, authorization, scalability, and high availability.

HOW THE SOLUTIONS SATISFY THE CASE STUDY RUBRIC

1. Understanding of Case – 25%

The proposed solutions clearly identify the business requirements, users, technical requirements, security requirements, scalability requirements, and availability requirements of each case.

2. Application of Concepts – 25%

Appropriate cloud technologies such as APIs, cloud storage, databases, virtual machines, Kubernetes, analytics services, authentication, VPNs, firewalls, and access-control mechanisms are mapped directly to the requirements.

3. Analysis – 20%

The solutions analyze the advantages and disadvantages of each architecture. Important trade-offs such as cloud dependency, operational cost, Kubernetes complexity, real-time synchronization, security, and scalability are considered.

4. Solution Design – 20%

Each solution provides a practical cloud architecture connecting clients, APIs, cloud services, storage, databases, analytics, and security mechanisms. The proposed architectures are scalable and feasible for real-world implementation.

5. Clarity – 10%

The solutions are organized into introduction, requirements, cloud concepts, architecture, security, scalability, advantages, trade-offs, and conclusion. This makes the technical design easy to understand and evaluate.

FINAL CONCLUSION

Cloud computing provides the infrastructure and services required to build scalable, secure, reliable, and cost-effective applications. The three case studies demonstrate different applications of cloud technology.

The Smart Retail system uses cloud computing for centralized inventory management and intelligent demand forecasting. The Document Collaboration system uses distributed cloud infrastructure and real-time APIs to support simultaneous editing and global collaboration. The Online Food Ordering system uses cloud APIs, virtual machines, Kubernetes, cloud storage, authentication, HTTPS, and RBAC to provide a scalable and secure application.

Therefore, cloud computing is suitable for all three scenarios because it provides on-demand resources, scalability, high availability, centralized services, security mechanisms, and the ability to integrate different applications and services in real time.